Hybrid Multiverse-based Parallel Computing Framework for Task Scheduling in Swarm Robotics

Mohmmadsadegh Mokhtari*, David Scalais†, Jeroen Famaey*
*University of Antwerp -IDLab, imec, Belgium
†University of Antwerp, Belgium

***Abstract*—Energy-aware task scheduling in large robotic swarms is challenging due to computational complexity, communication overhead, and limited onboard energy. Traditional centralized schedulers struggle to scale, while fully distributed approaches often lack global coordination. To address this, we propose a hybrid scheduling framework in which a central coordinator performs global task allocation using a Multi-Verse Optimizer (MVO), and the swarm itself participates in parallel local schedule refinement. After receiving their assigned task subsets, individual robots refine execution order using Particle Swarm Optimization (PSO) or Genetic Algorithms (GA), allowing computation to be distributed across the swarm rather than concentrated at the center. This significantly reduces central processing demand by offloading while adapting task execution to local energy conditions. The framework is implemented in a ROS–Docker environment with explicit energy-aware scheduling. Experimental results show that the approach reduces scheduling computation time by up to 250% and lowers mission makespan by 5–7% compared to six state-of-the-art methods, while improving overall energy efficiency and coordination. These findings demonstrate that combining centralized global insight with distributed parallel refinement enables scalable and energy-efficient swarm task scheduling suitable for large-scale deployments.**

I. Introduction

Swarm robotics coordinates many autonomous agents to perform cooperative tasks in domains such as environmental monitoring, search and rescue, precision agriculture, warehouse automation, and pickup-and-delivery (P&D) logistics [1]. A central challenge in these systems is task scheduling, which encompasses both task allocation and determining execution order under constraints of limited energy, intermittent communication, and onboard computational capacity. The complexity of this scheduling problem grows rapidly with swarm size and task volume, leading to potential performance degradation and reduced mission endurance.

Traditional centralized schedulers can generate globally consistent solutions [2], but they introduce a single-point computational bottleneck and require communication of complete schedule updates, which does not scale well for large swarms. Conversely, fully distributed approaches improve robustness and reduce communication load, yet may lack global coordination, resulting in redundant motion, workload imbalance, and elevated energy consumption [3]. Recent hybrid task allocation frameworks have therefore begun to split computation: a central entity determines which robot should handle each task, while each robot refines its own execution plan locally, reducing central load and distributing computation across the swarm [4].

Following this hybrid global–local architecture, we propose a scalable, energy-aware scheduling framework. At the global layer, we employ the Multi-Verse Optimizer (MVO) to compute an initial task allocation. MVO is selected over alternative global metaheuristics because of its demonstrated ability to maintain population diversity and avoid premature convergence in high-dimensional optimization landscapes [5]. It has also shown strong performance in large-scale scheduling and resource allocation problems [6], making it suitable for global workload distribution in swarms.

After allocation, computation is intentionally offloaded to the swarm itself: each robot refines the order of its assigned subtasks using lightweight Particle Swarm Optimization (PSO) or Genetic Algorithms (GA). PSO and GA are widely used for local search due to their fast convergence characteristics and low onboard computational overhead [7]. This distributed refinement allows robots to adapt their schedules to their own energy levels, task proximity, and local dynamic conditions. However, the central coordinator remains responsible for validating and approving the final task schedules, ensuring that the overall plan remains globally consistent and conflict-free before execution. By enabling parallel schedule refinement while retaining a global decision authority, the framework reduces central computational load and maintains coordinated behavior—an approach consistent with recent hybrid MRTA systems in which robots compute local bidding or insertion heuristics while a centralized unit maintains global coherence [4]. The proposed framework is implemented in a ROS–Docker environment, ensuring reproducibility and scalable deployment.

II. Related Work

Task scheduling in swarm robotics has been widely studied, with recent literature emphasizing the trade-offs between computational efficiency, scalability, and energy-awareness. Mokhtari et al. [8] present a centralized, energy-aware scheduling framework for heterogeneous robot swarms, formulating the problem as a sequence-dependent job shop with energy and charging constraints. Their Adaptive Particle Swarm Optimization (APSO) approach achieves high-quality solutions but suffers from the inherent limitations of centralization. Similarly, Cechinel et al. [9] develop a ROS-based

centralized warehouse scheduling system using an island-model GA, yet all computation remains centralized, creating a bottleneck for large-scale deployments.

Several recent studies have explored alternative optimization strategies to improve multi-robot task allocation efficiency. Fu and Wang [10] introduce a Swiss Round Selection GA for warehouse scheduling, which improves convergence and diversity maintenance but still assumes a centralized computational model.

Yuan et al. [11] combine K-means++ clustering with PSO for task allocation, reducing search space dimensionality and improving solution quality; however, their method is evaluated in static, small-scale settings and does not consider energy constraints. A recent systematic review by Rema et al. [12] highlights that most existing approaches still rely on centralized scheduling, with distributed methods receiving comparatively less attention.

Distributed methods have emerged as a promising alternative to address these scalability issues. Ferreira et al. [13] propose a distributed multi-stage optimization framework for heterogeneous teams using a distributed GA with knowledge sharing, improving scalability but lacking the integration of global exploration with fine-grained local refinement.

Hybrid metaheuristics have been increasingly explored to leverage the strengths of multiple algorithms. Abualigah et al. [14] combine the MVO with GA for cloud task scheduling, while Makhadmeh et al. [15] integrate MVO with Grey Wolf Optimizer (GWO) for multi-objective power scheduling. Although effective in their respective domains, these approaches remain centralized and are not tailored for dynamic, energy-constrained robotic swarms.

De Ryck et al. [4] introduce a hybrid task allocation strategy for AGV fleets where a central unit assigns tasks and robots locally update their routes using insertion heuristics. Their approach demonstrates the value of distributing part of the computation to the robots, reducing central overhead and improving scalability in constrained environments. However, the refinement step in their framework relies on deterministic heuristics, which can limit the quality of solutions when tasks are spatially diverse or when resource conditions such as battery state vary significantly between robots.

In contrast, our framework retains the same global allocation and local refinement structure but improves both flexibility and solution quality by using evolutionary optimizers (PSO or GA) at the robot level rather than heuristic insertion. This allows each robot to search a richer local schedule space and adapt dynamically to its own energy state and local environmental conditions. Moreover, while computation is distributed across the swarm, the central coordinator still validates the refined plans before execution, ensuring global coherence and preventing conflicts. As a result, our method preserves the scalability benefits of distributed computation while achieving higher-quality, energy-aware schedules suitable for large and heterogeneous swarm deployments.

Our work bridges these gaps by coupling centralized MVO-based global exploration with distributed PSO/GA-based local refinement. This architecture enables parallel execution and explicitly optimizes both completion time and energy usage, making it suitable for large-scale, real-world swarm robotic systems.

III. Energy-Aware Task Scheduling Problem

To evaluate the proposed algorithm, we adopt the fitness formulation introduced by Mokhtari et al. [8], which effectively addresses the time–cost trade-off optimization problem. This formulation aims to minimize the overall mission completion time while maintaining reliability under uncertain energy harvesting conditions. The detailed implementation of this fitness function is summarized in Algorithm 1, with further comprehensive explanations available in the original work.

Following the original formulation, we consider the sets of robots $\mathbb{R}$, tasks $\mathbb{O}$, and charging stations $\mathbb{C}$. Each robot $r \in \mathbb{R}$ has initial state variables, including energy level E^r_{start}, location p^r_{start}, and an assigned task queue. These form the foundational elements for the scheduling problem.

The robot's remaining energy after completing a task depends on whether it requires recharging. If the robot must recharge before starting task $o_i \in \mathbb{O}$, the energy level at task completion is:

$$\begin{aligned} E^r_{end}(o_i) = E^r_{max} + E^r_{harvest}(T^r_{start}(o_i), T^r_{end}(o_i)) \\ - E_{drive}(p_{c_i}, p^0_{o_i}) - E_{o_i} \end{aligned} \tag{1}$$

Here, E^r_{max} is the battery's maximum capacity, $E^r_{harvest}$ represents the energy harvested during task execution from start to end times, E_{drive} is the energy spent driving from the charging station location p_{c_i} to the task start location $p^0_{o_i}$, and E_{o_i} is the energy consumed performing the task.

If the robot has enough energy to complete the task without recharging, the remaining energy is computed as:

$$\begin{aligned} E^r_{end}(o_i) = E^r_{start} + E^r_{harvest}(T^r_{start}(o_i), T^r_{end}(o_i)) \\ - E_{drive}(p^1_{o_{i-1}}, p^0_{o_i}) - E_{o_i} \end{aligned} \tag{2}$$

where E^r_{start} is the robot's energy level at the start of the task, and $E_{drive}(p^1_{o_{i-1}}, p^0_{o_i})$ is the energy required to travel from the previous task location $p^1_{o_{i-1}}$ to the current task start location $p^0_{o_i}$.

The completion time for task o_i by robot r is given by:

$$T^r_{end}(o_i) = T^r_{start}(o_i) + T^r_{setup} + T_{o_i} \tag{3}$$

Here, T_{o_i} is the execution time of the task, and T^r_{setup} represents the travel (and possibly charging) time between consecutive tasks, defined as:

$$\begin{cases} T^r_{setup} = T_{drive}(p^1_{o_{i-1}}, p^0_{o_i}), \text{ if } C_r \neq \emptyset \\ T^r_{setup} = T_{drive}(p^1_{o_{i-1}}, p_c) \\ \qquad\qquad + T_{charge}(E^r_c, E_{max}) \\ \qquad\qquad + T_{drive}(p_c, p^0_{o_i}), \text{ if } C_r = \emptyset \end{cases} \tag{4}$$

where C_r indicates whether robot r has sufficient charge to proceed without recharging, p_c is the charging station location, and $T_{charge}(E_c^r, E_{max})$ is the charging duration required to replenish energy from current charge E_c^r to maximum E_{max}.

A. Objective

The objective is to minimize the makespan, the maximum task completion time across all robots and assigned tasks:

$$\min \left(\max_{r \in \mathbb{R}, o \in O_{r,m}} T_{end}^r(o) \right) \tag{5}$$

where $O_{r,m} \subseteq \mathbb{O}$ is the subset of tasks assigned to robot r for mission m. The scheduling and energy constraints guide the task assignment.

B. Constraints

1) **Deadline constraint:** Each task must complete before its deadline td_m:

$$\max_{o \in O_m} T_{end}^r(o) \leq td_m \tag{6}$$

2) **Energy sufficiency:** After task completion, the robot's energy must be enough to reach the nearest charging station safely:

$$E_{end}^r(o) \geq E_{r_c} \tag{7}$$

where E_{r_c} is the minimum energy reserve required for reaching the nearest charging station.

3) **Sequential execution:** Tasks must follow the predefined order, ensuring mission m_i finishes after m_{i-1}:

$$T_{end}^{m_i} > T_{end}^{m_{i-1}} \tag{8}$$

Algorithm 1: Fitness Function Evaluation [8]

Input: Particle set P^t representing robot-task assignments
Output: Fitness value F_{fitness}

```
for i ← 1 to m do
    Calculate task duration T_{o_i} and energy consumption E_{o_i} for the assigned robot in P_i^t;
    Predict harvested energy E_harvest during the task period;
    if C_r = ∅ (insufficient energy) then
        Insert a charging task before o_i;
        Update T_end^r and E_end^r using Eq. (1) and Eq. (3);
    else
        Update T_end^r and E_end^r using Eq. (2) and Eq. (3);
    if any constraint (6)–(8) is violated then
        F_fitness ← ∞;
        break
Compute F_fitness from Eq. (5);
```

IV. Proposed MVO-GA and MVO-PSO Algorithms

We adapt the MVO [16] for global optimization of the energy-aware scheduling problem. The system inputs include the set of robots $\mathbb{R}$, the set of tasks $\mathbb{O}$, and the set of charging stations $\mathbb{C}$, along with initial state variables for each robot such as energy levels, locations, and assigned task queues.

Key parameters are population size S, search space bounds lb and ub, wormhole existence probability bounds $WEP_{\min}$ and $WEP_{\max}$, and an exploitation accuracy parameter p.

Our hybrid computational architecture combines a centralized MVO-based global search with distributed, parallel local refinement using GA or PSO at each robot. The central unit explores the solution space and distributes promising candidates to robots, which locally improve these solutions. The final solutions are aggregated centrally. Fig. 1 illustrates this hybrid optimization framework. More details on MVO can be found in [16].

A. Global Search via MVO

Let:

$$\mathbf{X}_i^{(g)} = [x_{i1}^{(g)}, x_{i2}^{(g)}, \ldots, x_{iD}^{(g)}]$$

denote the ith candidate solution (or universe) at generation g, where D is the dimensionality of the solution space and S is the population size. Each candidate solution $\mathbf{X}_i^{(g)}$ encodes the complete task–robot assignment and scheduling decisions, with each component $x_{ij}^{(g)}$ representing a specific assignment variable or scheduling parameter.

Let $f(\mathbf{X}_i^{(g)})$ represent the fitness of universe i, indicating the quality of the task–robot assignment.

The MVO explores the solution space by simulating interactions between parallel universes. This work uses only the exploration phase (white hole mechanism) for global search, while local exploitation is handled separately using a distributed genetic algorithm.

a) White Hole Mechanism (Exploration): During each generation, candidate solutions probabilistically inherit components from better-performing solutions through the white hole mechanism. This encourages exploration of new regions influenced by fitter universes:

$$x_{ij}^{(g+1)} = \begin{cases} x_{kj}^{(g)}, & \text{if rand}() < \text{NIW}_i^{(g)} \\ x_{ij}^{(g)}, & \text{otherwise} \end{cases} \tag{9}$$

Here, $x_{ij}^{(g)}$ is the jth component of the ith universe at generation g, and $x_{kj}^{(g)}$ is the corresponding component from a more fit universe k. The term $\text{NIW}_i^{(g)}$ (Normalized Inflation Weight) is computed by normalizing the fitness values of all universes to the range $[0, 1]$, representing the relative quality of universe i at generation g. It determines the probability that components from universe i are passed to other universes during exploration. Finally, rand() is a uniformly distributed random number in the interval $[0, 1]$.

This mechanism maintains diversity by replacing small parts (genes) of solutions with those from high-quality ones, helping the algorithm explore the search space widely before

fine-tuning. In the MVO, this happens through the white hole operator, which randomly copies variables from better solutions to others. Since it only involves simple random choices and variable copying without heavy calculations, this step is computationally light compared to the full optimization process [17], [18].

b) Global Evaluation Budget: The total number of evaluations across all generations is:

$$E_{\text{global}} = S \times G \tag{10}$$

where G is the total number of generations in the global MVO process, and S is the population size. This product defines the total number of fitness evaluations during global search.

B. Distributed Local Refinement

After completing G global generations, the top S solutions are distributed for local optimization. Each robot $r_k \in \mathbb{R}$ receives:

$$S_k = \left\lfloor \frac{S}{|\mathbb{R}|} \right\rfloor \tag{11}$$

to process in parallel. The remaining $(S \bmod |\mathbb{R}|)$ solutions are assigned one each to the first robot r_1 through $r_{(S \bmod |\mathbb{R}|)}$ to ensure load balancing.

C. Local Optimization via GA

Each robot optimizes its solutions using a GA [19] with the following components:

Selection: Roulette wheel selection with probability

$$p_i = \frac{f(\mathbf{x}_i)}{\sum_j f(\mathbf{x}_j)} \tag{12}$$

Crossover: Uniform or two-point crossover applied to parent chromosomes with probability p_{cross}.

Mutation: Each gene x_i in an offspring is mutated with probability p_{mut}:

$$x_i \leftarrow \begin{cases} \text{mutated value with probability } p_{\text{mut}} \\ x_i \text{ with probability } 1 - p_{\text{mut}} \end{cases}$$

D. Local Optimization via PSO

For continuous task assignments, each robot applies PSO [20] to locally refine its schedule.

The velocity update for the i-th particle at iteration $t+1$ is computed as:

$$\begin{aligned} v_i^{(t+1)} = \omega v_i^{(t)} + c_1 r_1 (pbest_i - x_i^{(t)}) \\ + c_2 r_2 (gbest - x_i^{(t)}) \end{aligned} \tag{13}$$

where $v_i^{(t)}$ and $x_i^{(t)}$ are the velocity and position of particle i at iteration t, ω is the inertia weight controlling momentum, c_1 and c_2 are acceleration coefficients, r_1 and r_2 are random values uniformly sampled from $[0, 1]$, $pbest_i$ is the personal best position found by the particle, and $gbest$ is the global best position found by the swarm.

The particle's position is then updated by adding the new velocity:

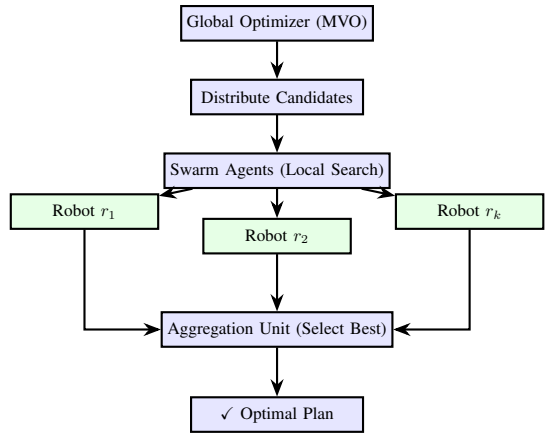

Fig. 1: Hybrid optimization framework: MVO with distributed refinement.

$$\mathbf{x}_i^{(t+1)} = \mathbf{x}_i^{(t)} + \mathbf{v}_i^{(t+1)} \tag{14}$$

This iterative update helps particles explore the solution space effectively, balancing exploration and exploitation to converge toward high-quality local solutions for task scheduling.

E. Final Aggregation and Decision

After local refinement, each robot sends its best solution X_k^* to the central unit. The global solution is selected as:

$$X^* = \arg \min_{X_k^* \in \mathcal{R}_{\text{local}}} f(X_k^*) \tag{15}$$

where $\mathcal{R}_{\text{local}}$ is the set of refined results from all robots.

V. Simulation Results

In this section, we present the results comparing centralized and distributed computation approaches, as well as the impact of integrating the MVO for exploration.

Simulation performance is assessed via two metrics: simulation time, the total runtime including all algorithmic operations and communication overhead, and completion time, the duration reported by the fitness function for finishing assigned tasks. Experiments ran on a host with a 2.7 GHz dual-core Intel Core i5 and 32 GB RAM, with Docker limited to 4 CPUs and 4 GB RAM. Each data point reports the mean of 10 independent runs to capture stochastic variability.

Our system employs a modular distributed architecture using ROS and Docker, consisting of a coordinator node, a task publisher, and multiple robot nodes. The coordinator manages task requests and collects robot states via ROS services, enabling informed decision-making. Tasks are asynchronously published by the task publisher via ROS topics, while robot states are synchronously obtained via ROS services, as illustrated in Fig. 2.

Our scenario investigates the effect of increasing the number of tasks (M) on completion and simulation times, with the number of robots fixed at $N = 5$ and $N = 20$. Initial battery

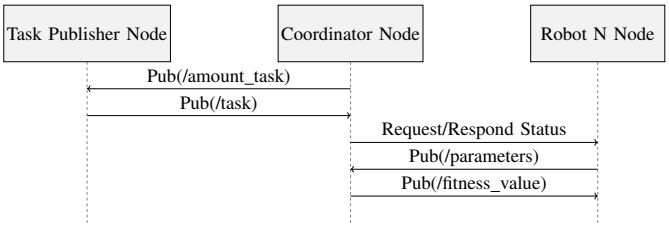

Fig. 2: Communication sequence between Task Publisher, Coordinator, and Robot nodes.

charges are randomly selected between 20% and 80% of the nominal battery capacity $E_{max} = 20\,Wh$. The charging time for a full battery is set as $T_{charge}(0, E_{max}) = 1850\,s$. For an initial battery charge E_{init}, the charging time is proportional to the remaining capacity and calculated as:

$$T_{charge}(E_{init}, E_{max}) = 1850 \times \frac{E_{max} - E_{init}}{E_{max}} \quad . \tag{16}$$

We compare eight algorithm configurations: HMVOGA, DGA, HMVOPSO, DPSO, CMVOGA, CMVOPSO, CGA, and CPSO. Among these, HMVOGA and HMVOPSO represent our proposed distributed MVO-based optimization approaches, while the remaining six algorithms serve as baseline methods for comparison. In this notation, the prefix *C* denotes centralized computation (single-node processing), *D* denotes distributed computation (parallel execution across multiple nodes), and *H* denotes a hybrid configuration in which the *MVO* operates in a centralized manner while the *GA* or *PSO* performs distributed local optimization. The *MVO* component indicates that the Multi-Verse Optimizer is employed during the initial global search stage to enhance exploration efficiency. The *GA* and *PSO* components correspond to the local refinement stage, where each robot independently optimizes its assigned subtasks.

As shown in Fig. 3 and Fig. 5, simulation times across all eight methods are consistently lower in distributed and hybrid configurations. In particular, HMVOGA and HMVOPSO achieve up to a 50% reduction in simulation time for $N = 5$ robots and up to 250% for $N = 20$ robots compared to their centralized counterparts. This improvement was greater than initially expected, as distributed MVO-based methods were hypothesized to incur additional communication and synchronization overhead, as it is depicted to some extent in CMVOPSO and CMVOGA simulation time results. The observed efficiency gain can be attributed to the smaller effective search space per agent and enhanced information flow enabled by distributed parallel execution.

While these reductions in simulation time are significant, the primary objective remains minimizing overall task completion time. As illustrated in Fig. 4 and Fig. 6, the hybrid configurations HMVOGA and HMVOPSO compete closely with DGA to achieve the lowest completion times, reducing task completion by approximately 5–7% on average compared to the other methods.

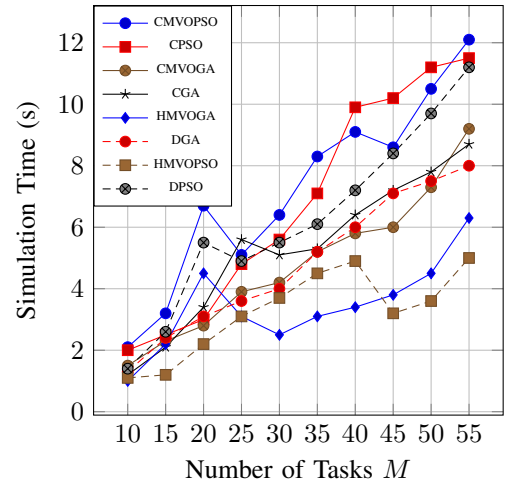

Fig. 3: Simulation Time vs. Number of Tasks: Distributed methods consistently reduce simulation time by up to 50% compared to their centralized pair approaches for $N = 5$.

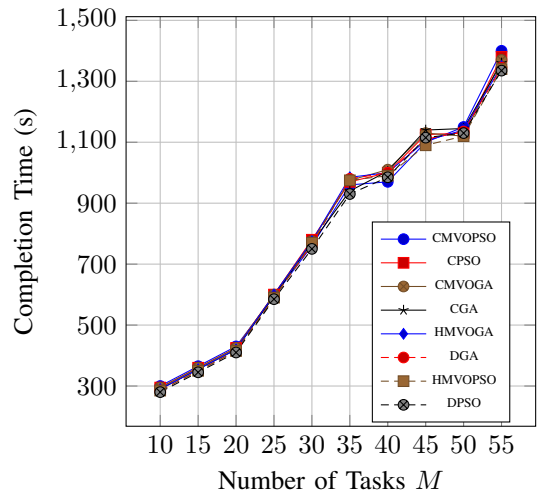

Fig. 4: Completion Time vs. Number of Tasks: MVO-based distributed methods consistently achieve the lowest completion times, improving performance by about 5–7% compared to other approaches for $N = 5$.

VI. Conclusion

In conclusion, our novel hybrid computing algorithm significantly reduces both completion and simulation times compared to other methods, achieving up to 250% faster simulations and 5–7% lower completion times. This demonstrates its suitability for swarm-based robotic systems, enhancing autonomy, precision, and computational efficiency. Although tested on complex scheduling tasks, the approach applies broadly to swarm optimization problems. Future work will explore energy-aware task distribution and cluster-based system evaluations to further improve performance.

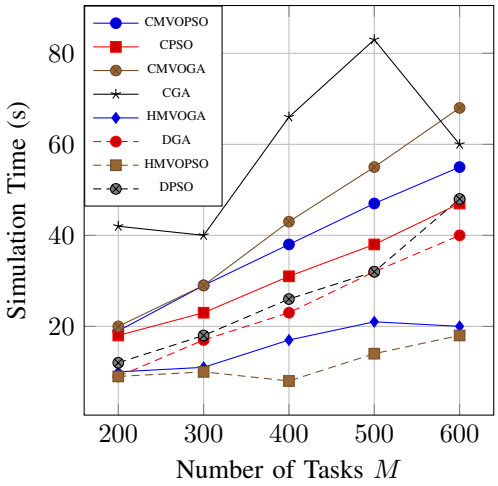

Fig. 5: Simulation Time vs. Number of Tasks: Distributed methods consistently reduce simulation time by up to 250% compared to their centralized pair approaches for $N = 20$.

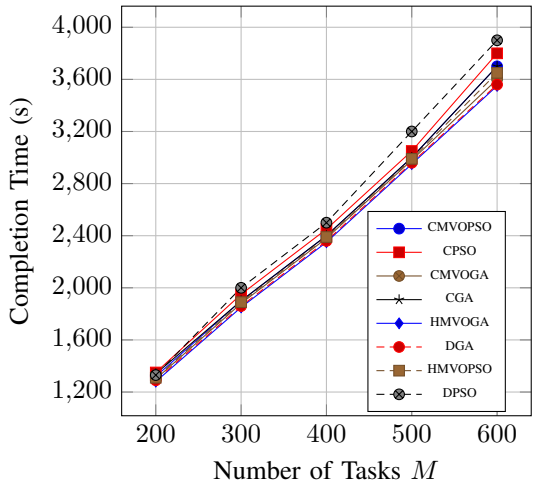

Fig. 6: Completion Time vs. Number of Tasks: MVO-based distributed methods consistently achieve the lowest completion times, improving performance by about 5–7% compared to other approaches for $N = 20$.

Acknowledgment

This work was supported by the EU Horizon Europe Grant 101093046 (OpenSwarm) and the Flemish Government FWO Project G019722N (LOCUSTS). Views expressed are those of the authors and do not reflect the European Commission's position.

References

[1] H. Hamann, *Swarm Robotics: A Formal Approach*. Springer, 2018.

[2] M. B. Dias, R. Zlot, N. Kalra, and A. Stentz, "Market-based multirobot coordination: A survey and analysis," *Proceedings of the IEEE*, vol. 94, no. 7, pp. 1257–1270, 2006.

[3] M. Shili, "Energy-aware swarm robotics in smart microgrids using distributed control and metaheuristics," *Electronics*, vol. 14, no. 21, p. 4210, 2025.

[4] M. De Ryck, G. Van den Berghe, and P. Vansteenwegen, "Decentral task allocation for industrial agv-systems with resource constraints," *Journal of Manufacturing Systems*, vol. 59, pp. 310–319, 2021.

[5] J. J. Jui, I. Molla, A. Ahmad, and S. Hettiarachchi, "Recent advances and applications of the multi-verse optimiser algorithm: A survey from 2020 to 2024," *Archives of Computational Methods in Engineering*, 2025.

[6] P. V. H. Son and D. N. Trinh, "Solving large-scale discrete time–cost trade-off problem using hybrid multi-verse optimizer model," *Scientific Reports*, vol. 13, no. 1987, 2023.

[7] E. V. Altay, O. Altay, and Y. Özçevik, "A comparative study of metaheuristic optimization algorithms for solving real-world engineering design problems," *Computer Modeling in Engineering & Sciences*, vol. 139, no. 1, pp. 1039–1094, 2023.

[8] M. Mokhtari, "Energy-aware multi-robot task scheduling using metaheuristic optimization methods for ambiently-powered robot swarms," *Robotics and Autonomous Systems*, vol. 186, 2025.

[9] A. Cechinel and E. Pieri, "Centralized multi-robot logistic system: An approach using the island model genetic algorithm as task scheduler," *International Journal of Advanced Robotic Systems*, vol. 21, Oct. 2024.

[10] X. Fu and Y. Wang, "Swiss round selection genetic algorithm for multi-robot task scheduling in warehouses," *Engineering Applications of Artificial Intelligence*, 2024.

[11] Y. Yuan, P. Yang, H. Jiang, and T. Shi, "A multi-robot task allocation method based on the synergy of the k-means++ algorithm and the particle swarm algorithm," *Biomimetics*, vol. 9, no. 11, p. 694, 2024.

[12] C. Rema, P. Costa, M. Silva, and E. J. S. Pires, "Task scheduling with mobile robots—a systematic literature review," *Robotics*, vol. 14, no. 6, p. 75, 2025.

[13] B. Arbanas Ferreira, T. Petrović, and S. Bogdan, "Distributed mission planning of complex tasks for heterogeneous multi-robot teams," *arXiv preprint arXiv:2109.10106*, 2021.

[14] L. Abualigah and M. Alkhrabsheh, "Amended hybrid multi-verse optimizer with genetic algorithm for solving task scheduling problem in cloud computing," *The Journal of Supercomputing*, vol. 78, no. 1, pp. 740–765, Jan. 2022.

[15] A. Makhadmeh, H. Abasi, and M. A. Al-Betar, "Hybrid multi-verse optimizer with grey wolf optimizer for power scheduling problem in smart home using iot," *The Journal of Supercomputing*, vol. 78, pp. 11 794–11 829, June 2022.

[16] S. Mirjalili, S. M. Mirjalili, and A. Hatamlou, "Multi-verse optimizer: a nature-inspired algorithm for global optimization," *Neural Computing and Applications*, vol. 27, no. 2, pp. 495–513, Feb. 2016.

[17] S. Mirjalili, C. B. Kalayci, and A. Biswas, *A Brief Tutorial on Optimization Problems, Optimization Algorithms, Meta-Heuristics, and Swarm Intelligence*. Springer, Cham, 2023, pp. 3–14.

[18] L. Abualigah, "Multi-verse optimizer algorithm: a comprehensive survey of its results, variants, and applications," *Neural Computing and Applications*, vol. 32, no. 16, pp. 12 381–12 401, 2020.

[19] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor, MI: University of Michigan Press, 1975.

[20] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proceedings of the IEEE International Conference on Neural Networks (ICNN)*. IEEE, 1995.